

primera

WITH signal_data AS (

SELECT

case_flag,

CASE

WHEN deterioration_acceleration > 0

THEN 1 ELSE 0

END AS sig_deterioration,

CASE

WHEN utilization_change_vs_baseline > 0.037

THEN 1 ELSE 0

END AS sig_utilization,

CASE

WHEN delinquency_share_change_vs_baseline > 0

THEN 1 ELSE 0

END AS sig_delinquency_share,

CASE

WHEN adverse_recent_3m > 0

THEN 1 ELSE 0

END AS sig_adverse,

CASE

WHEN balance_change_vs_baseline > 31798.99

THEN 1 ELSE 0

END AS sig_extreme_balance

FROM bu_frd.alert_case_control_3m_2025

WHERE utilization_change_vs_baseline IS NOT NULL

),

combinations AS (

SELECT

case_flag,

CONCAT(

'D', CAST(sig_deterioration AS STRING),

'_U', CAST(sig_utilization AS STRING),

'_DS', CAST(sig_delinquency_share AS STRING),

'_A', CAST(sig_adverse AS STRING),

'_B', CAST(sig_extreme_balance AS STRING)

```

    ) AS signal_combination,

    sig_deterioration
    + sig_utilization
    + sig_delinquency_share
    + sig_adverse
    + sig_extreme_balance
    AS behavioral_signal_count

FROM signal_data
)

SELECT
    signal_combination,
    behavioral_signal_count,

    SUM(
        CASE WHEN case_flag = 1 THEN 1 ELSE 0 END
    ) AS case_count,

    SUM(
        CASE WHEN case_flag = 0 THEN 1 ELSE 0 END
    ) AS control_count,

    ROUND(
        100.0 *
        SUM(CASE WHEN case_flag = 1 THEN 1 ELSE 0 END)
        / 141,
        2
    ) AS pct_of_cases,

    ROUND(
        100.0 *
        SUM(CASE WHEN case_flag = 0 THEN 1 ELSE 0 END)
        / 2728,
        2
    ) AS pct_of_controls

FROM combinations

GROUP BY
    signal_combination,
    behavioral_signal_count

ORDER BY
    behavioral_signal_count DESC,
    case_count DESC,
    control_count DESC;

```

segunda

WITH base AS (

SELECT

case_flag,

CASE

WHEN deterioration_acceleration > 0

THEN 1 ELSE 0

END AS sig_deterioration,

CASE

WHEN utilization_change_vs_baseline > 0.037

THEN 1 ELSE 0

END AS sig_utilization,

CASE

WHEN delinquency_share_change_vs_baseline > 0

THEN 1 ELSE 0

END AS sig_delinquency_share,

CASE

WHEN adverse_recent_3m > 0

THEN 1 ELSE 0

END AS sig_adverse,

CASE

WHEN balance_change_vs_baseline > 31798.99

THEN 1 ELSE 0

END AS sig_extreme_balance

FROM bu_frd.alert_case_control_3m_2025

WHERE utilization_change_vs_baseline IS NOT NULL

),

scored AS (

SELECT

case_flag,

sig_deterioration,

sig_utilization,

sig_delinquency_share,

sig_adverse,

sig_extreme_balance,

(

```

        sig_deterioration
        + sig_utilization
        + sig_delinquency_share
        + sig_adverse
        + sig_extreme_balance
    ) AS behavioral_signal_count

FROM base
),

combinations AS (

SELECT
    case_flag,
    behavioral_signal_count,

    CONCAT(
        'D', CAST(sig_deterioration AS STRING),
        '_U', CAST(sig_utilization AS STRING),
        '_DS', CAST(sig_delinquency_share AS STRING),
        '_A', CAST(sig_adverse AS STRING),
        '_B', CAST(sig_extreme_balance AS STRING)
    ) AS signal_combination

FROM scored
)

SELECT
    signal_combination,
    behavioral_signal_count,

    SUM(
        CASE
            WHEN case_flag = 1
            THEN 1 ELSE 0
        END
    ) AS case_count,

    SUM(
        CASE
            WHEN case_flag = 0
            THEN 1 ELSE 0
        END
    ) AS control_count,

    ROUND(
        100.0 *
        SUM(CASE WHEN case_flag = 1 THEN 1 ELSE 0 END)

```

```

        / 141,
        2
    ) AS pct_of_cases,

    ROUND(
        100.0 *
        SUM(CASE WHEN case_flag = 0 THEN 1 ELSE 0 END)
        / 2728,
        2
    ) AS pct_of_controls

FROM combinations

GROUP BY
    signal_combination,
    behavioral_signal_count

ORDER BY
    behavioral_signal_count DESC,
    case_count DESC,
    control_count DESC;

test

WITH base AS (

    SELECT
        case_flag,

        CASE WHEN deterioration_acceleration > 0
            THEN 1 ELSE 0 END AS d,

        CASE WHEN utilization_change_vs_baseline > 0.037
            THEN 1 ELSE 0 END AS u,

        CASE WHEN delinquency_share_change_vs_baseline > 0
            THEN 1 ELSE 0 END AS ds,

        CASE WHEN adverse_recent_3m > 0
            THEN 1 ELSE 0 END AS a,

        CASE WHEN balance_change_vs_baseline > 31798.99
            THEN 1 ELSE 0 END AS b

    FROM bu_frd.alert_case_control_3m_2025

    WHERE utilization_change_vs_baseline IS NOT NULL
),

```

```
x AS (
    SELECT
        case_flag,
        d + u + ds + a + b AS signal_count
    FROM base
)
```

```
SELECT
    case_flag,
    signal_count,
    COUNT(*) AS observation_count
```

```
FROM x
```

```
GROUP BY
    case_flag,
    signal_count
```

```
ORDER BY
    signal_count,
    case_flag DESC;
```

ahora con lo de python

1. Cardinalidad

```
cardinality = pd.DataFrame({
    "level": [
        "exact_ip",
        "/64",
        "/56",
        "/48"
    ],
    "n_unique": [
        df["ip"].nunique(),
        df["prefix_64"].nunique(),
        df["prefix_56"].nunique(),
        df["prefix_48"].nunique()
    ]
})
```

```
cardinality
```

```
print(
```

```

    "IPs por /64:",
    df.groupby("prefix_64")["ip"].nunique().describe()
)

```

```

print(
    "IPs por /56:",
    df.groupby("prefix_56")["ip"].nunique().describe()
)

```

```

print(
    "IPs por /48:",
    df.groupby("prefix_48")["ip"].nunique().describe()
)

```

2. Persistencia temporal

```

def persistence_stats(data, column):
    temp = (
        data.groupby(column)["week"]
        .nunique()
    )

    return pd.Series({
        "entities": len(temp),
        "mean_weeks": temp.mean(),
        "median_weeks": temp.median(),
        "pct_1_week": (temp == 1).mean() * 100,
        "pct_ge_4_weeks": (temp >= 4).mean() * 100,
        "pct_ge_8_weeks": (temp >= 8).mean() * 100,
        "pct_ge_12_weeks": (temp >= 12).mean() * 100,
    })

```

```

persistence = pd.DataFrame({
    "exact_ip": persistence_stats(df, "ip"),
    "prefix_64": persistence_stats(df, "prefix_64"),
    "prefix_56": persistence_stats(df, "prefix_56"),
    "prefix_48": persistence_stats(df, "prefix_48"),
}).T

```

persistence

3. Número de IP distintas por prefijo

```

def ips_per_prefix(data, prefix_col):
    return (

```

```

data.groupby(prefix_col)["ip"]
.nunique()
.describe(
    percentiles=[
        0.25,
        0.5,
        0.75,
        0.90,
        0.95,
        0.99
    ]
)
)

```

```

ips_per_prefix(df, "prefix_64")
ips_per_prefix(df, "prefix_56")
ips_per_prefix(df, "prefix_48")

```

4. Agregar attack_count semanal para cada nivel

```

weekly_ip = (
    df.groupby(
        ["ip", "week"]
    )
    .size()
    .reset_index(name="attack_count")
)

```

```
weekly_ip.head()
```

```

weekly_64 = (
    df.groupby(
        ["prefix_64", "week"]
    )
    .size()
    .reset_index(name="attack_count")
)

```

```

weekly_56 = (
    df.groupby(
        ["prefix_56", "week"]
    )
    .size()
    .reset_index(name="attack_count")
)

```

```

weekly_48 = (
    df.groupby(
        ["prefix_48", "week"]
    )
    .size()
    .reset_index(name="attack_count")
)

```

5. Comparar correlación lag-1

```

def lag1_correlation(data, entity_col):
    temp = data.sort_values(
        [entity_col, "week"]
    ).copy()

    temp["lag1"] = (
        temp.groupby(entity_col)["attack_count"]
        .shift(1)
    )

    valid = temp.dropna(
        subset=["attack_count", "lag1"]
    )

    if len(valid) == 0:
        return np.nan

    return valid[
        ["attack_count", "lag1"]
    ].corr().iloc[0, 1]

```

```

lag_correlations = pd.DataFrame({
    "level": [
        "exact_ip",
        "/64",
        "/56",
        "/48"
    ],
    "lag1_corr": [
        lag1_correlation(
            weekly_ip,
            "ip"
        ),
        lag1_correlation(

```

```

        weekly_64,
        "prefix_64"
    ),

    lag1_correlation(
        weekly_56,
        "prefix_56"
    ),

    lag1_correlation(
        weekly_48,
        "prefix_48"
    ),
]
})

```

lag_correlations

6. Mejor versión: comprobar que el lag sea realmente una semana anterior

```

def lag1_calendar_correlation(data, entity_col):
    temp = data.sort_values(
        [entity_col, "week"]
    ).copy()

    temp["previous_week"] = (
        temp.groupby(entity_col)["week"]
        .shift(1)
    )

    temp["lag1"] = (
        temp.groupby(entity_col)["attack_count"]
        .shift(1)
    )

    temp["week_difference"] = (
        temp["week"]
        - temp["previous_week"]
    ).dt.days

    valid = temp[
        temp["week_difference"] == 7
    ].dropna()

    if len(valid) == 0:
        return np.nan

```

```

return valid[
    ["attack_count", "lag1"]
].corr().iloc[0, 1]

lag_correlations_calendar = pd.DataFrame({
    "level": [
        "exact_ip",
        "/64",
        "/56",
        "/48"
    ],
    "lag1_corr": [
        lag1_calendar_correlation(
            weekly_ip,
            "ip"
        ),
        lag1_calendar_correlation(
            weekly_64,
            "prefix_64"
        ),
        lag1_calendar_correlation(
            weekly_56,
            "prefix_56"
        ),
        lag1_calendar_correlation(
            weekly_48,
            "prefix_48"
        ),
    ]
})

```

lag_correlations_calendar

7. Comparar lag 1, 2, 4 y 12

```

def lag_correlations(data, entity_col, lags=(1, 2, 4, 8, 12)):
    temp = data.sort_values(
        [entity_col, "week"]
    ).copy()

    result = {}

    for lag in lags:

```

```

temp[f"lag{lag}"] = (
    temp.groupby(entity_col)["attack_count"]
    .shift(lag)
)

temp[f"week_lag{lag}"] = (
    temp.groupby(entity_col)["week"]
    .shift(lag)
)

difference = (
    temp["week"]
    - temp[f"week_lag{lag}"]
).dt.days

valid = temp[
    difference == lag * 7
]

corr = valid[
    ["attack_count", f"lag{lag}"]
].corr().iloc[0, 1]

result[f"lag{lag}"] = corr

return result

comparison = pd.DataFrame({
    "exact_ip": lag_correlations(
        weekly_ip,
        "ip"
    ),

    "prefix_64": lag_correlations(
        weekly_64,
        "prefix_64"
    ),

    "prefix_56": lag_correlations(
        weekly_56,
        "prefix_56"
    ),

    "prefix_48": lag_correlations(
        weekly_48,
        "prefix_48"
    ),

```

```
}).T
```

comparison

8. Un score preliminar que sí puede ayudarte

```
summary = pd.DataFrame({
    "level": [
        "exact_ip",
        "/64",
        "/56",
        "/48"
    ],

    "n_entities": [
        df["ip"].nunique(),
        df["prefix_64"].nunique(),
        df["prefix_56"].nunique(),
        df["prefix_48"].nunique()
    ],

    "median_weeks": [
        persistence.loc[
            "exact_ip",
            "median_weeks"
        ],

        persistence.loc[
            "prefix_64",
            "median_weeks"
        ],

        persistence.loc[
            "prefix_56",
            "median_weeks"
        ],

        persistence.loc[
            "prefix_48",
            "median_weeks"
        ],
    ],

    "pct_ge_12_weeks": [
        persistence.loc[
            "exact_ip",
            "pct_ge_12_weeks"
        ],
```

```

persistence.loc[
    "prefix_64",
    "pct_ge_12_weeks"
],

persistence.loc[
    "prefix_56",
    "pct_ge_12_weeks"
],

persistence.loc[
    "prefix_48",
    "pct_ge_12_weeks"
],
],

"lag1_corr": [
    lag_correlations_calendar.loc[
        lag_correlations_calendar["level"]
        == "exact_ip",
        "lag1_corr"
    ].iloc[0],

    lag_correlations_calendar.loc[
        lag_correlations_calendar["level"]
        == "/64",
        "lag1_corr"
    ].iloc[0],

    lag_correlations_calendar.loc[
        lag_correlations_calendar["level"]
        == "/56",
        "lag1_corr"
    ].iloc[0],

    lag_correlations_calendar.loc[
        lag_correlations_calendar["level"]
        == "/48",
        "lag1_corr"
    ].iloc[0],
]
})

summary

```